Bootstrap Integration

Estimated time needed: 30 minutes

In this lab, you will practice integrating bootstrap into django templates.

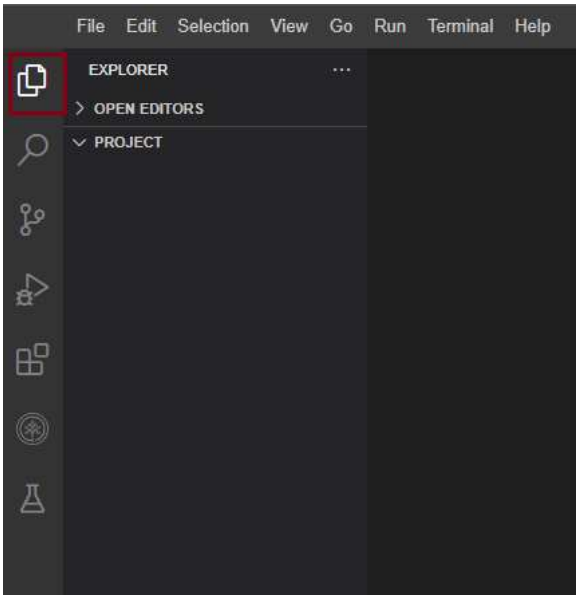
Learning Objectives

- Integrate Bootstrap front-end library into your Django templates

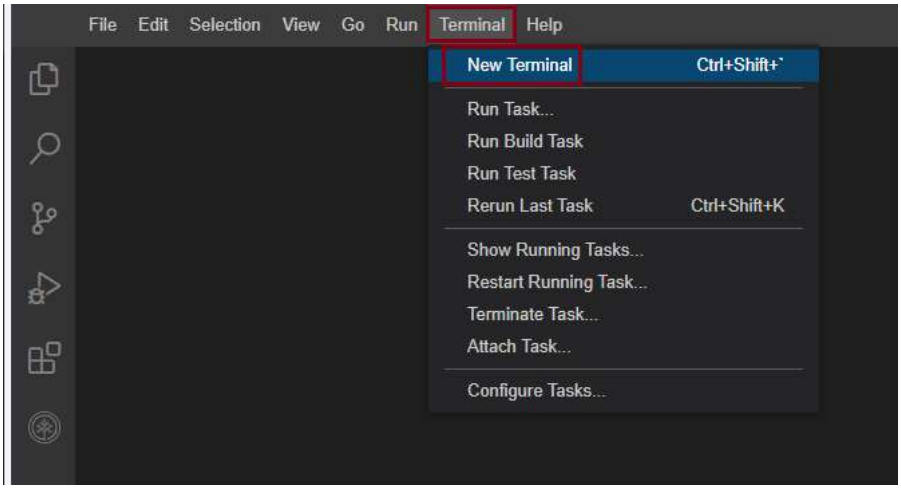
Working with files in Cloud IDE

If you are new to Cloud IDE, this section will show you how to create and edit files, which are part of your project, in Cloud IDE.

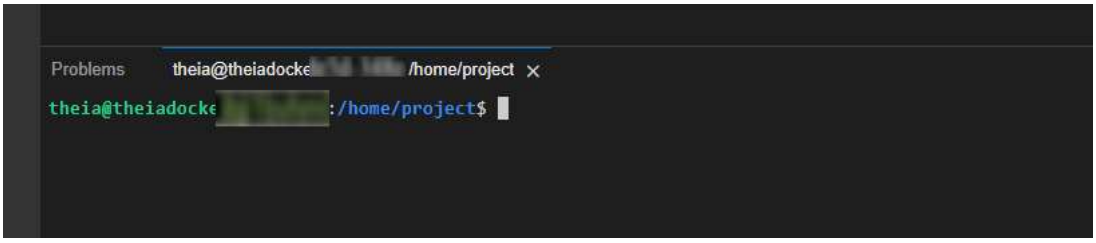
To view your files and directories inside Cloud IDE, click on this files icon to reveal it.



Click on `New`, and then `New Terminal`.



This will open a new terminal where you can run your commands.



Concepts covered in the lab

1. `Bootstrap`: A popular open-source framework that provides a set of CSS and JavaScript components for building responsive and mobile-first web applications.
2. `Template`: A Django-specific file that defines the structure and presentation of a web page. It typically contains a mixture of HTML, CSS, and Django template language code.
3. `Django template language`: A specialized template language provided by Django that allows dynamic content rendering, looping, conditional statements, and more.
4. `Bootstrap Components`: Pre-designed UI elements provided by Bootstrap, such as buttons, forms, navigation bars, and modals, that can be easily integrated into web pages.
5. `Bootstrap Grid System`: A responsive layout system provided by Bootstrap that divides the page into a grid of rows and columns, allowing for easy alignment and arrangement of content.

Import an `onlinecourse` App Template and Database

If the terminal was not open, go to `Terminal > New Terminal` and make sure your current Theia directory is `/home/project`.

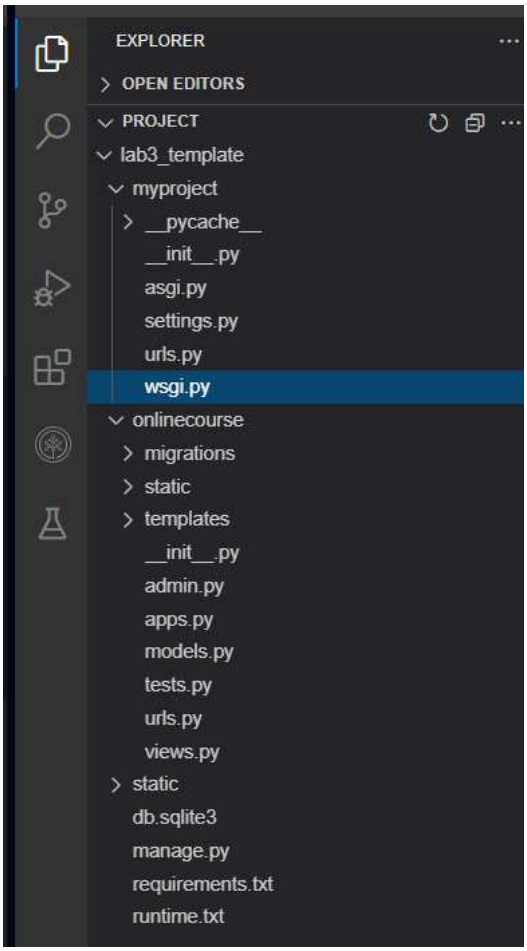
- Run the following command-lines to download a code template for this lab

```
1
2
3
```

```
wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-CD0251EN-SkillsNetwork/labs/m5_django_advanced/lab3_template.zip"
unzip lab3_template.zip
rm lab3_template.zip
```

Copied! Executed!

Your Django project should look like the following:



- `cd` to the project folder:

```
1
```

```
cd lab3_template
```

Copied! Executed!

- Install these must-have packages and setup the environment.

```
1
2
3
4
5
```

```
pip install --upgrade distro-info
pip3 install --upgrade pip==23.2.1
pip install virtualenv
virtualenv djangoenv
source djangoenv/bin/activate
```

Copied! Executed!

- Install the necessary Python packages.

```
1
```

```
pip install -r requirements.txt
```

Copied! Executed!

The `requirements.txt` contains all necessary Python packages for you to run this lab.

Next activate the models for an `onlinecourse` app.

- Perform migrations to create necessary tables:

```
1
```

```
python3 manage.py makemigrations
```

Copied! Executed!

- and run migration to activate models for the `onlinecourse` app.

```
1
```

```
python3 manage.py migrate
```

Copied! Executed!

Now let's test the imported `onlinecourse` app.

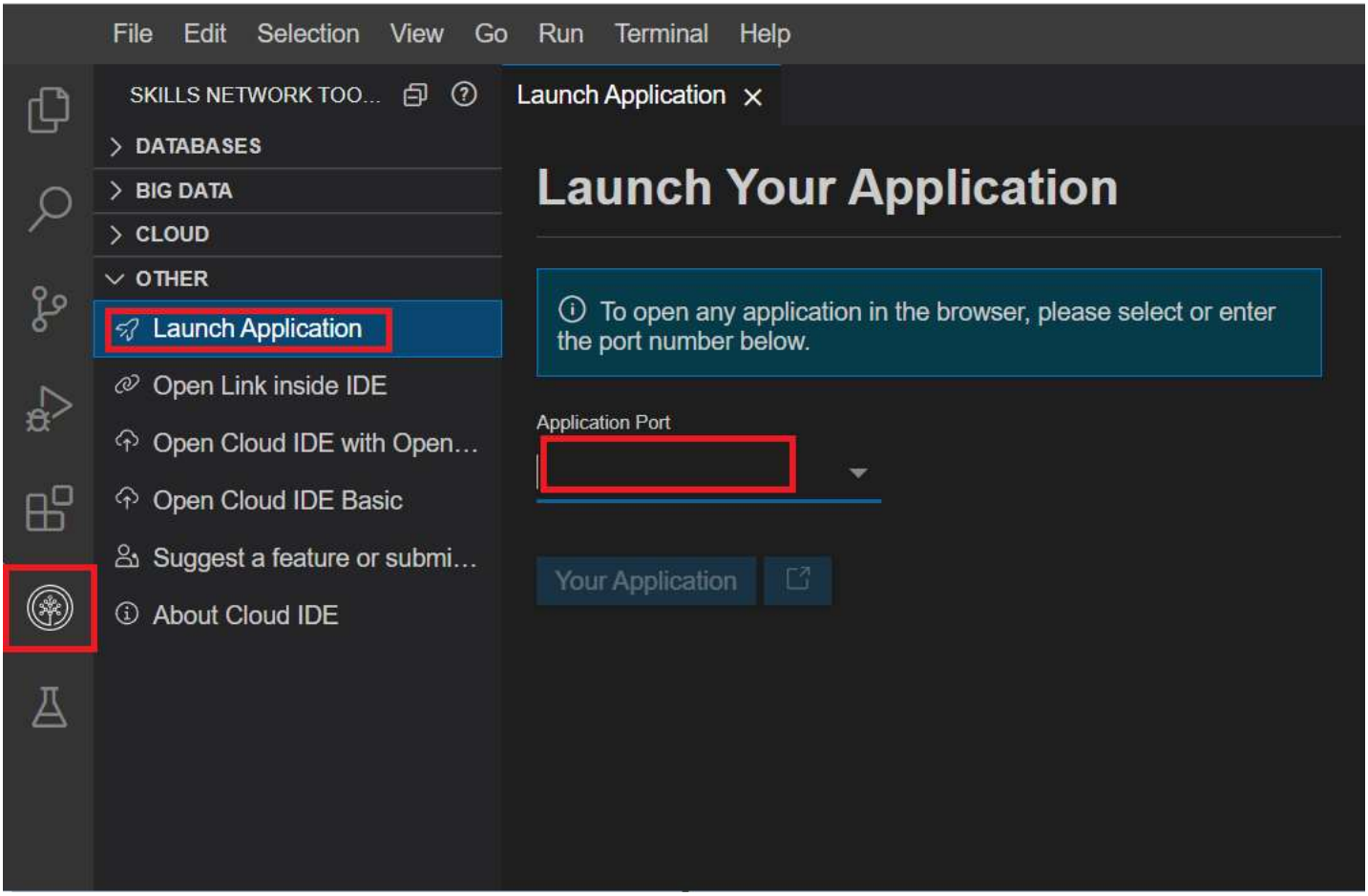
- Start the development server:

```
1
```

```
python3 manage.py runserver
```

Copied! Executed!

- Click on the Skills Network button on the left, it will open the **"Skills Network Toolbox"**. Then click the **Other** then **Launch Application**. From there you should be able to enter the port `8000` and launch.



When the browser tab opens, add the `/onlinecourse` path and your full URL should look like the following

```
https://userid-8000.theiadocker-1.proxy.cognitiveclass.ai/onlinecourse
```

Now you should see the `onlinecourse` app started, and all the pages are rendered from unpolished Django templates without any CSS.

In this lab, you will be learning how to use Bootstrap to stylize those pages.

Integrate Bootstrap

Adding a minimal Bootstrap CSS file to a Django template is actually very easy.

- Open `onlinecourse/templates/onlinecourse/course_list.html`, link a Bootstrap CSS file `bootstrap.min.css` to `<head>...</head>` element

```
1. 1
2. 2
3. 3
4. 4
5. 5
6. 6
```

```
1. <head>
2.     {% load static %}
3.     <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
4.     <meta charset="UTF-8">
5.     <title>Online Courses</title>
6. </head>
```

Copied!

Here we used `maxcdn` CDN (Content Distribution Network) for delivering `bootstrap.min.css` file to our templates. In addition to `maxcdn`, there are many other popular CDNs you may use such as `StackPath`.

- Repeat the above steps to update the `<head>` of all template files in `onlinecourse/templates/onlinecourse` folder.

Create a Navigation Bar

The first UI improvement is adding a navigation bar to help users accessing information in our app.

- Open `onlinecourse/templates/onlinecourse/course_list.html`, and add an empty light themed navigation bar `<nav>`with a navigation bar header (as the app brand):

```
1. 1
2. 2
3. 3
4. 4
```

```
5
6
7
```

```
<nav class="navbar navbar-light bg-light">
    <div class="container-fluid">
        <div class="navbar-header">
            <a class="navbar-brand" href="#">Online Course</a>
        </div>
    </div>
</nav>
```

Copied!

The content `<div>` of the navigation bar `<nav>` uses a `.container-fluid` Bootstrap CSS class to make sure the bar content takes the full width of the page.

Then we added a `<div>` with `.navbar-header` class to define header with a simple link styled with `.navbar-brand` to highlight the app name.

Refresh the main course list page and you should see a light theme navigation bar with a header called `Online Course`.

Next, let's move the user sign-in and sign-up area to the navigation bar.

- Remove the existing `<ul>` list of user authentication related elements such as user name and password input fields.
- Under `<div class="navbar-header">`, add a new list styled with classes `nav navbar-nav navbar-right` to create a navigation bar item and be aligned to the right.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

```
<ul class="nav navbar-nav navbar-right">
    {% if user.is_authenticated %}
    <li>
        <a class="btn btn-link" href="#">{{ user.first_name }}({{ user.username }})</a>
        <a class="btn btn-link" href="{% url 'onlinecourse:logout' %}">Logout</a>
    </li>
    {% else %}
    <li>
        <form class="form-inline" action="{% url 'onlinecourse:login' %}" method="post">
            {% csrf_token %}
            <div class="input-group">
                <input type="text" class="form-control" placeholder="Username" name="username" >
                <input type="password" class="form-control" placeholder="Password" name="psw" >
                <button class="btn btn-primary" type="submit">Login</button>
                <a class="btn btn-link" href="{% url 'onlinecourse:registration' %}">Sign Up</a>
            </div>
        </form>
    </li>
    {% endif %}
</ul>
```

Copied!

The completed `<nav>` HTML element should look like the following code snippet:

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
```

```
27
```

```
<nav class="navbar navbar-light bg-light">
    <div class="container-fluid">
        <div class="navbar-header">
            <a class="navbar-brand" href="#">Online Course</a>
        </div>
        <ul class="nav navbar-nav navbar-right">
            {% if user.is_authenticated %}
            <li>
                <a class="btn btn-link" href="#">{{ user.first_name }}({{ user.username }})</a>
                <a class="btn btn-link" href="{% url 'onlinecourse:logout' %}">Logout</a>
            </li>
            {% else %}
            <li>
                <form class="form-inline" action="{% url 'onlinecourse:login' %}" method="post">
                    {% csrf_token %}
                    <div class="input-group">
                        <input type="text" class="form-control" placeholder="Username" name="username" >
                        <input type="password" class="form-control" placeholder="Password" name="psw" >
                        <button class="btn btn-primary" type="submit">Login</button>
                        <a class="btn btn-link" href="{% url 'onlinecourse:registration' %}">Sign Up</a>
                    </div>
                </form>
            </li>
            {% endif %}
        </ul>
    </div>
</nav>
```

Copied!

Refresh the page and you should see a nice navigation bar with user login and registration elements integrated.

Coding Practice: Create Navigation Bar for All Templates

Copy/paste the navigation bar your created in the previous step to `course_detail.html`, `user_login.html`, `user_registration.html`.

- For all templates other than the course list template, change the navigation bar header link pointing to Online Course, i.e., page generated by `popular_course_list`

▼ Click here to see solution

```
1
```

```
<a class="navbar-brand" href="{% url 'onlinecourse:popular_course_list' %}">Online Course</a>
```

Copied!

Stylize Login and Registration Pages

Next, let's stylize the login and registration pages.

- Open the `onlinecourse/templates/onlinecourse/user_login.html`, find the root `<div>` with comment `<!--Style root div with .container class -->` and stylize it:

```
1
```

```
<div class="container">
```

Copied!

Bootstrap `.container` class adds some paddings to the `<div>`.

- Find the second level `<div>` element with comment `<!--Style second div with .form-group class -->` and stylize it:

```
1
```

```
<div class="form-group">
```

Copied!

Bootstrap `.form-group` class builds the structure for a form.

- Find two `<input>` elements with comment `<!--Style input with .form-control class -->` and stylize it with:

```
1
```

```
1. <input ... class = "form-control" ...>
```

Copied!

Bootstrap `.form-control` class stylizes the elements in a form.

- Find a `<div>` with comment `<!--Style the message paragraph with .alert and .alert-warning class -->`. This `<div>` is designed to show warning messages such as user already exists or user password is not correct. Stylize it with:

```
1. 1
1. <div class="alert alert-warning">
```

Copied!

- Find the submit `<button>` with comment `<!--Style button with .btn and .btn-primary class -->`. Stylize it with

```
1. 1
1. <button class="btn btn-primary" ... >
```

Copied!

Once you have stylized above elements, the login form should look like the following:

```
1. 1
2. 2
3. 3
4. 4
5. 5
6. 6
7. 7
8. 8
9. 9
10. 10
11. 11
12. 12
13. 13
14. 14
15. 15
16. 16
17. 17
18. 18
```

```
1. <form action="{% url 'onlinecourse:login' %}" method="post">
2.   {% csrf_token %}
3.   <div class="container"> <!--Style root div with .container class -->
4.     <h1>Login</h1>
5.     <div class="form-group"> <!--Style second div with .form-group class -->
6.       <label for="username"><b>User Name</b></label>
7.       <input id="username" class = "form-control" type="text" placeholder="Enter User Name: " name="username" required> <!--Style input with .form-
8.       <label for="psw"><b>Password</b></label>
9.       <input id="psw" class = "form-control" type="password"  placeholder="Enter Password: " name="psw" required> <!--Style input with .form-contro
10.     </div>
11.     {% if message %}
12.       <div class="alert alert-warning"> <!--Style the message paragraph with .alert and .alert-warning class -->
13.         {{ message }}
14.       </div>
15.     {% endif %}
16.     <button class="btn btn-primary" type="submit">Login</button> <!--Style button with .btn and .btn-primary class -->
17.   </div>
18. </form>
```

Copied!

- Next, let's go to `https://userid-8000.theiadocker-1.proxy.cognitiveclass.ai/onlinecourse/login` and check the result:

and your login page should look like the following:

Coding Practice: Stylize the Registration Page

- Stylize the `user_registration.html` by following the examples in `user_login.html` page
  and the comments in the template.

▶ Click here to see solution

Stylize the Course List using Card and CardDeck classes

Next, let's stylize the main content of course list page.

For the course list, we want to organize each course as a card with an image, title, and description and put them onto a card deck.

- Open `onlinecourse/templates/onlinecourse/course_list.html`, find the content root level `<div>` with comment `<!--Style root div with .container class -->`
  and stylize it with:

```
1
<div class="container">
```

Copied!

- Find the second level card deck `<div>` with comment `<!--Style second div with .card-deck class -->` and stylize it with:

```
1
<div class="card-deck">
```

Copied!

- Find the third level card `<div>` with comment `<!--Style third level div with .card class -->` and stylize it with:

```
1
<div class="card" ... >
```

Copied!

- Find card image `<image>` with comment `<!--Style card image with .card-img-left class -->` and stylize it with:

```
1
<img class="card-img-left" ... >
```

Copied!

- Find the card body `<div>` class with comment `<!--Style root div with .card-body and .bg-light class -->` and stylize it with:

```
1
<div class="card-body bg-light">
```

Copied!

- Find the card title `<h5>` and `<span>` with comment `<!--Style h5 with .card-title and span with .text-success class -->` and stylize them with:

```
1
<h5 class="card-title"> ... <span class="text-success"> ...
```

Copied!

- Find the card text `<p>` with comment `<!--Style card description with .card-text class -->` and stylize it with:

```
1
<p class="card-text">
```

Copied!

- Find the enrollment submission button `<input>` with comment `<!--Style Enroll button with .btn and .btn-primary class -->` and stylize it with:

```
1
<input class="btn btn-primary" >
```

Copied!

Now your course list is stylized into a course card deck

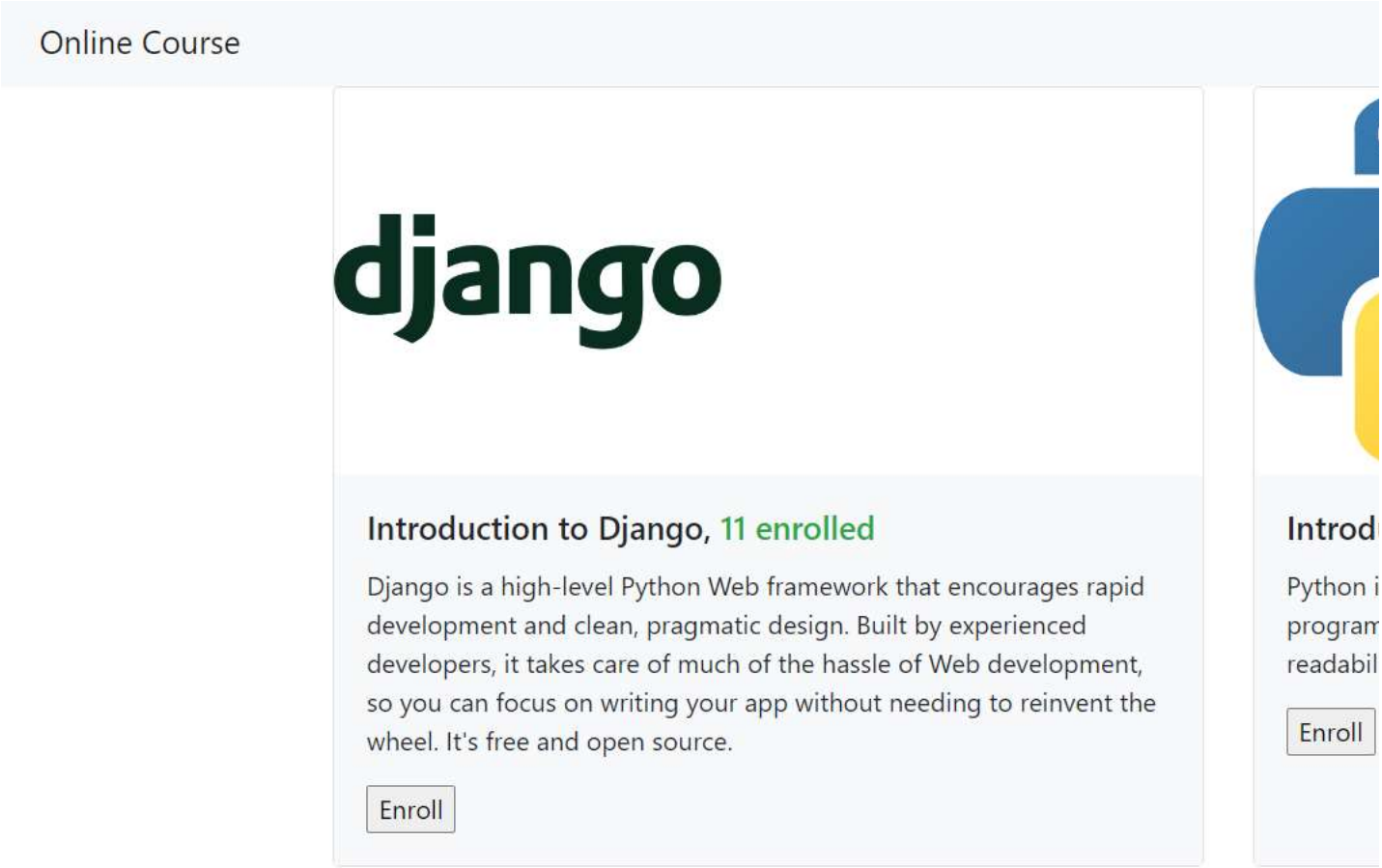
```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
```

```
{% if course_list %}
    <div class="container"> <!--Style root div with .container class -->
        <div class="card-deck"> <!--Style second div with .card-deck class -->
            {% for course in course_list %}
            <div class="card" style="width: 36rem;"> <!--Style third level div with .card class -->
                <img class="card-img-left" src="{{MEDIA_URL}}/{{ course.image }}" width="240px" height="240px"
                         alt="Course image"> <!--Style card image with .card-img-left class -->
                <div class="card-body bg-light"> <!--Style root div with .card-body and .bg-light class -->
                    <h5 class="card-title">{{ course.name }}, <span class="text-success"> <!--Style h5 with .card-title and span with .text-success
                                {{ course.total_enrollment}} enrolled</span></h5>
                    <p class="card-text">{{ course.description}}</p> <!--Style card description with .card-text class -->
                    <form action="{% url 'onlinecourse:enroll' course.id %}" method="post">
                        {% csrf_token %}
                        <input class="btn btn-primary" type="submit"
                                value="Enroll"> <!--Style Enroll button with .btn and .btn-primary class -->
                    </form>
                </div>
            </div>
            {% endfor %}
        </div>
    </div>
{% else %}
    <p>No courses are available.</p>

{% endif %}
```

Copied!

Let's open `https://userid-8000.theiadocker-1.proxy.cognitiveclass.ai/onlinecourse` and check the fully stylized course card deck page:



Coding Practice: Stylize Course Details Page

- Stylize `onlinecourse/templates/onlinecourse/course_details.html` by making each lesson as a card and the lesson list as a vertical card column.

▶ Click here to see solution

Summary

During this lab, you have gained hands-on experience in incorporating Bootstrap into your Django templates. You have also familiarized yourself with various frequently used Bootstrap CSS classes like .nav, .container, .card-deck, .card, and more.

Author(s)

Yan Luo

Changelog

Date	Version	Changed by	Change Description
14-Dec-2020	1.0	Yan Luo	Initial version created
5-Jul-2023	1.1	K Sundararajan	Updated instructions and copyright year as 2023
11-Jul-2023	1.2	Pallavi Rai	Updated instructions based on SME review
26-Jul-2023	1.3	K Sundararajan	Updates done as per QA review

© IBM Corporation 2023. All rights reserved.